

Codificação Para Front-end

19

2026

SENAI - CRTI

Form Validation JavaScript

The screenshot shows a web form titled "Form Validation" with the following fields and error messages:

- Full Name:** Input field containing "Piero Bruto".
- Email Address:** Input field containing "codingpage@igmail.com".
- Password:** Input field with a red border and error message "Enter your password".
- Birth Date:** Date picker with a red border and error message "Select your date of birth".
- Gender:** Dropdown menu with a red border and error message "Select your gender".

A green "Submit" button is located at the bottom of the form.

Eventos e Validação de Formulários

Transformando o e-commerce em uma aplicação verdadeiramente interativa — capturando ações do usuário, validando dados e respondendo em tempo real com JavaScript.

Situação de Aprendizagem

Onde estamos no projeto?

Nas aulas anteriores, construímos um e-commerce que **exibe produtos dinamicamente** usando manipulação do DOM. Os dados chegam, os cards aparecem na tela — mas o usuário ainda não consegue *interagir de verdade* com a aplicação.

Não há validação de formulários, não há resposta a cliques, não há feedback visual. É como uma vitrine sem porta de entrada.

 Hoje vamos abrir essa porta: implementar **eventos e validações**, tornando o projeto confiável e responsivo às ações do usuário.

○ que o projeto ainda não faz

- Validar campos de formulário antes do envio
- Responder a cliques e interações do usuário
- Exibir mensagens de erro ou sucesso
- Impedir ações inválidas ou indesejadas
- Manipular listas dinamicamente

Capacidades da Aula

O que você vai aprender hoje



Trabalhar com Eventos

Entender o modelo de eventos do JavaScript e como ouvir ações do usuário como cliques, digitação e envio de formulários.



`preventDefault()`

Usar `preventDefault()` para controlar o comportamento padrão do navegador e assumir o controle total do fluxo.



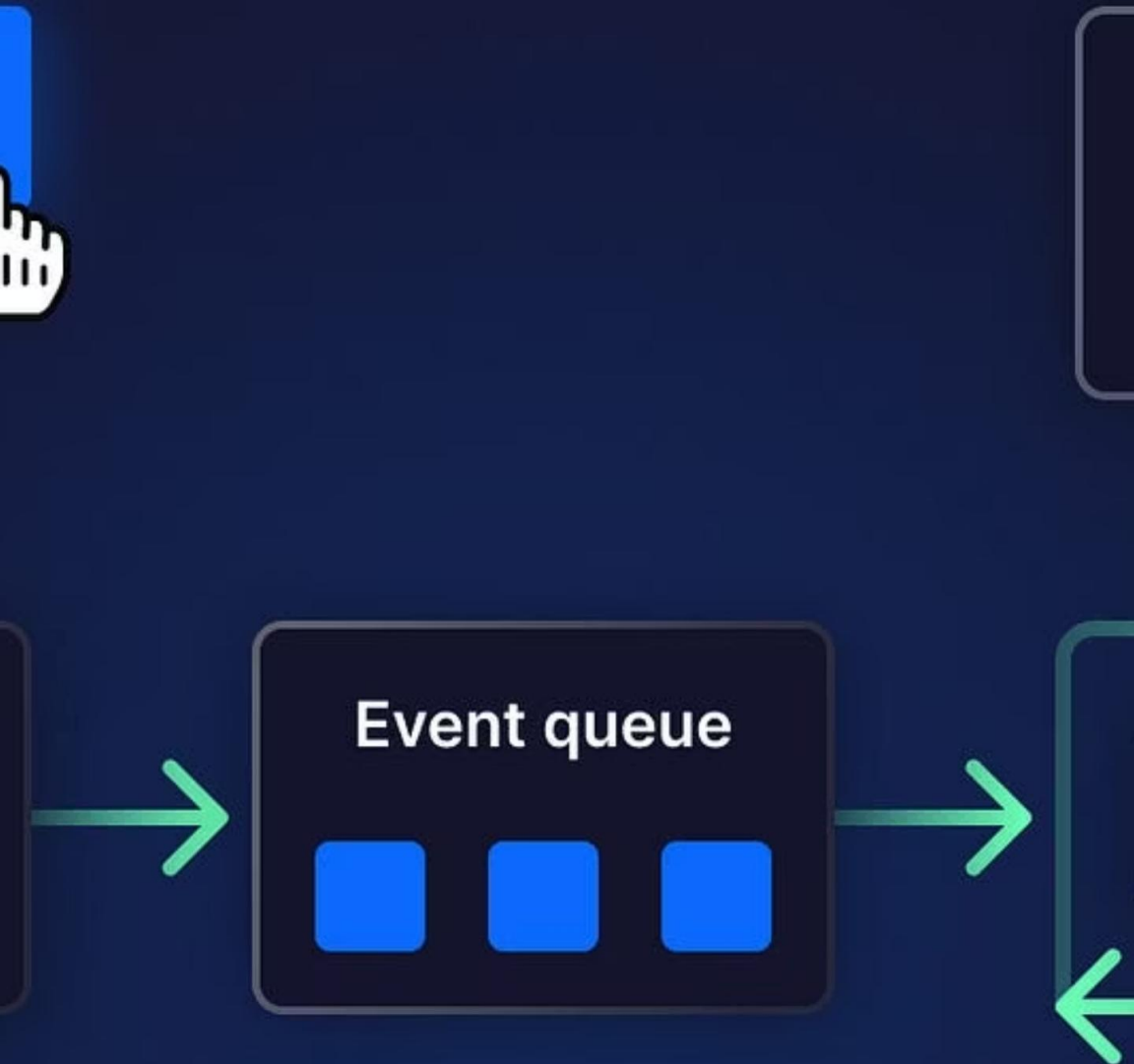
Objeto Event

Manipular o objeto `event` para acessar informações detalhadas sobre cada interação capturada pelo navegador.



Validações Customizadas

Criar validações próprias para campos de nome, e-mail e senha, com mensagens de feedback claras ao usuário.



O que são Eventos em JavaScript?

Um **evento** é qualquer ação que ocorre no navegador — um clique, uma tecla pressionada, o envio de um formulário, o movimento do mouse. O JavaScript permite que você *escute* esses momentos e execute código em resposta a eles.

Pense nos eventos como **gatilhos**: o usuário faz algo → o navegador dispara um evento → o seu código reage.

Eventos são a base da interatividade em qualquer aplicação web moderna. Sem eles, páginas seriam apenas documentos estáticos.

- ❏ O modelo de eventos do JavaScript segue o padrão **Observer**: você registra funções que serão chamadas automaticamente quando determinadas ações acontecerem.

Tipos de Eventos Essenciais

Conhecer os principais tipos de eventos é o primeiro passo para construir interfaces reativas. Veja os mais usados no desenvolvimento front-end:



click

Disparado ao clicar em um elemento. Usado em botões, links, cards e qualquer elemento clicável.



submit

Disparado quando um formulário é enviado. Fundamental para capturar e validar dados antes do envio.



input

Disparado a cada caractere digitado em um campo. Permite validação e feedback em tempo real.



change

Disparado quando o valor de um campo muda e perde o foco. Útil para selects e checkboxes.

addEventListener: Ouvindo Eventos

O método `addEventListener` é a forma moderna e recomendada de registrar eventos. Ele permite **múltiplos ouvintes** no mesmo elemento sem sobrescrever os anteriores — algo impossível com o antigo `onclick`.

Sintaxe básica:

```
elemento.addEventListener(  
  'evento',    // tipo do evento  
  callback,    // função executada  
  opcoes       // opcional  
);
```

- ✔ Boa prática: sempre prefira `addEventListener` em vez de atributos HTML como `onclick=""`. Separa HTML de JavaScript e facilita a manutenção.

Exemplo prático no e-commerce:

```
// Seleciona o botão "Adicionar ao Carrinho"  
const btnCarrinho = document.querySelector(  
  '#btn-carrinho'  
);  
  
// Ouve o evento de clique  
btnCarrinho.addEventListener('click', function(event)  
{  
  console.log('Produto adicionado!');  
  atualizarCarrinho();  
});  
  
// Com arrow function (mais moderno)  
btnCarrinho.addEventListener('click', (e) => {  
  exibirNotificacao('Item adicionado ao carrinho ✓');  
});
```

O Objeto **event**

O **event** (**ou e**) em JavaScript é o objeto criado automaticamente quando um evento acontece na página — como um clique, digitação, envio de formulário, etc. Ele carrega **todas as informações sobre o que aconteceu**.

preventDefault(): Assumindo o Controle

Por padrão, alguns eventos possuem comportamentos automáticos do navegador: formulários recarregam a página ao serem enviados, links navegam para outra URL. O método `preventDefault()` **cancela esse comportamento padrão**, permitindo que você assuma o controle total.

```
const formulario = document.querySelector('#form-  
login');  
  
formulario.addEventListener('submit', (e) => {  
  // Cancela o recarregamento da página  
  e.preventDefault();  
  
  // Agora temos controle total!  
  const email = document.querySelector('#email').value;  
  const senha = document.querySelector('#senha').value;  
  
  if (validarDados(email, senha)) {  
    realizarLogin(email, senha);  
  } else {  
    exibirErros();  
  }  
});
```



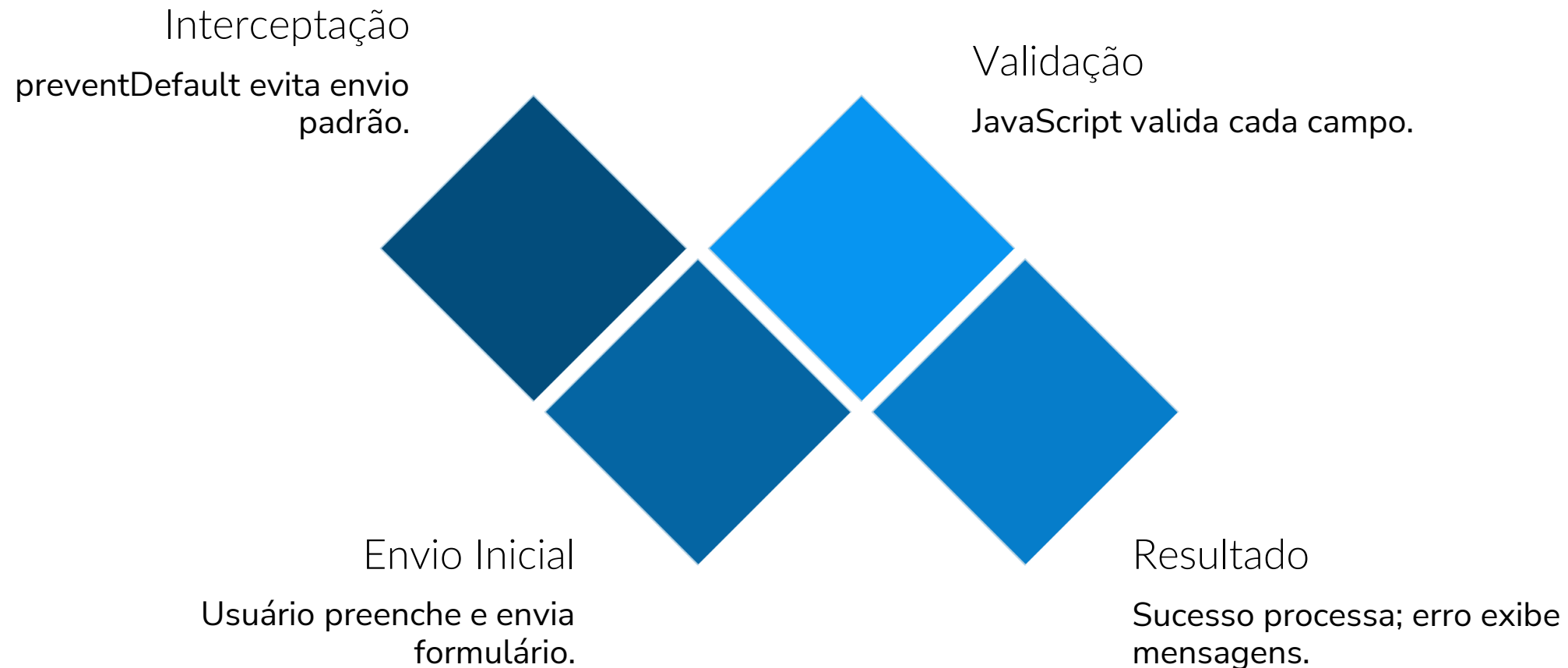
Sem `preventDefault()`, o formulário recarrega a página ao ser enviado, perdendo todos os dados e estados da aplicação.



O `preventDefault()` deve ser chamado **no início** do callback, antes de qualquer lógica, para garantir que o comportamento padrão seja sempre cancelado.

Validação de Formulários com JavaScript

A validação via JavaScript oferece **controle total** sobre as regras de negócio, feedback personalizado e uma experiência de usuário muito superior à v
nativa do HTML.



Esse fluxo garante que apenas dados válidos cheguem ao servidor, protegendo a integridade da aplicação e melhorando a experiência do usuário com mensagens claras e imediatas.

Feedback Visual ao Usuário

Uma boa validação não basta — o usuário precisa **entender o que está errado** e como corrigir. O feedback visual transforma uma experiência frustrante em uma orientação clara e amigável.

Mensagem de Erro

Exibida abaixo do campo inválido, em vermelho, com instrução objetiva de como corrigir.

Mensagem de Sucesso

Exibida após validação bem-sucedida, confirmando que a ação foi concluída corretamente.

Validação em Tempo Real

Feedback imediato enquanto o usuário digita, usando o evento `input`.

```
// Feedback de sucesso
function exibirSucesso(mensagem) {
  const div = document.createElement('div');
  div.className = 'alerta-sucesso';
  div.textContent = mensagem;

  // Insere no topo do formulário
  form.insertBefore(div, form.firstChild);

  // Remove após 4 segundos
  setTimeout(() => div.remove(), 4000);
}

// Validação em tempo real no campo email
const campoEmail =
  document.querySelector('#email');

campoEmail.addEventListener('input', (e) => {
  if (validarEmail(e.target.value)) {
    campoEmail.style.borderColor = '#28a745'; // Verde
    mostrarErro('email', '');
  } else {
    campoEmail.style.borderColor = '#dc3545'; // Vermelho
  }
});
```

Validação de Formulário

Capturando o envio do formulário

Quando clicamos em **Enviar**, o navegador dispara o evento **submit**.

Com JavaScript, podemos interceptar esse envio e validar os dados antes.

```
const form =  
  document.getElementById("formulario");  
  
const mensagem =  
  document.getElementById("mensagem");  
  
form.addEventListener("submit", function(event) {  
  event.preventDefault();});
```

 Estamos ouvindo o envio do formulário e impedindo o recarregamento automático da página.

Pegando os valores digitados

Lendo dados do usuário

Precisamos capturar o que foi digitado nos campos.

Usamos `.value`.

```
const nome =  
  document.getElementById("nome").value;  
  
const email =  
  document.getElementById("email").value;
```

 Agora o JavaScript consegue ler o nome e o email digitados.

Validando campos

Conferindo se os campos foram preenchidos

Aqui criamos uma regra:

Se algum campo estiver vazio → erro.

Se estiver preenchido → sucesso.

```
if (nome === "" || email === "") {  
    mensagem.textContent =  
        "Preencha todos os campos!";  
  
    mensagem.style.color = "red";  
} else {  
    mensagem.textContent =  
        "Formulário enviado com sucesso!";  
  
    mensagem.style.color = "green";  
}
```

 Estamos dando feedback visual imediato ao usuário.

Introdução à Lista de Tarefas

Criando interação real com eventos

Agora vamos construir algo interativo:
uma lista onde o usuário adiciona tarefas.

```
const inputTarefa =  
  document.getElementById("nova-tarefa");  
  
const botaoAdicionar =  
  document.getElementById("adicionar");  
  
const lista =  
  document.getElementById("lista-tarefas");
```

 Selecionamos os elementos que vamos manipular.

localStorage

Salvando dados no navegador

localStorage guarda informações mesmo após fechar a página.

```
let tarefas =  
  JSON.parse(  
    localStorage.getItem("tarefas")  
  ) || [];
```

 Se já existirem tarefas salvas, carregamos. Senão começamos com array vazio.

Renderizando tarefas

Mostrando itens na tela

```
function renderizarTarefas() {  
  lista.innerHTML = "";  
}  
  
tarefas.forEach((tarefa, index) => {  
  const li =  
    document.createElement("li");  
  
  li.textContent = tarefa;  
  lista.appendChild(li);  
});
```

 Percorremos o array e criamos elementos HTML dinamicamente..

Botão remover

Eventos em elementos criados dinamicamente

```
btn.addEventListener("click", () => {  
    tarefas.splice(index, 1);  
    salvar();  
});
```

 Cada botão sabe qual item remover.

Salvando alterações

Persistindo dados

```
function salvar() {  
  localStorage.setItem(  
    "tarefas",  
    JSON.stringify(tarefas)  
  );  
  
  renderizarTarefas();  
}
```

 Salva no navegador e atualiza a tela.

Adicionar tarefa

Escutando clique do botão

```
botaoAdicionar.addEventListener("click", () => {  
  const nova = inputTarefa.value;  
  
  if (nova !== "") {  
    tarefas.push(nova);  
    inputTarefa.value = "";  
    salvar();  
  }  
});
```

 Pegamos o texto digitado, adicionamos no array e renderizamos novamente.

Inicialização

Carregando a aplicação

```
renderizarTarefas();
```

 Quando a página abre, tudo já aparece automaticamente.

Armadilhas que Todo Dev Front-End Encontra



Esquecer o `preventDefault()`

O formulário recarrega a página antes de qualquer validação.
Sempre a primeira linha do callback de submit!



Não verificar campos vazios

Usar `value.length > 0` não basta — use `value.trim()`
para remover espaços em branco.



Registrar eventos múltiplas vezes

Adicionar `addEventListener` dentro de loops sem cuidado
gera múltiplos listeners no mesmo elemento.



Selecionar elemento antes de existir

Scripts no `<head>` sem `defer` tentam selecionar elementos
que ainda não foram renderizados pelo HTML.

❌ Dica extra: ao debugar eventos, use `console.log(e)` dentro do callback para inspecionar o objeto event completo no DevTools.

O que Construimos Hoje

Eventos

`click`, `submit`, `input`, `change` — cada um com seu propósito específico na construção de UIs interativas.

`addEventListener`

A forma moderna de registrar ouvintes de eventos, suportando múltiplos callbacks e separando HTML de JavaScript.

`preventDefault()`

Controle total sobre o comportamento padrão do navegador, essencial para formulários modernos com SPA.

Validação

Funções customizadas para cada campo, com feedback visual imediato e mensagens claras ao usuário.

To-Do List

Projeto completo unindo DOM dinâmico, eventos de clique, manipulação de lista e persistência opcional.

 Na **Aula 15**, vamos expandir esses fundamentos para construir o carrinho de compras completo, com estado gerenciado, atualização dinâmica e persistência real com `localStorage`.

Referências e Recursos para Estudar

Documentação Oficial

- **MDN Web Docs** — `EventTarget.addEventListener()`
`developer.mozilla.org`
- **MDN Web Docs** — `Event.preventDefault()`
Documentação completa com exemplos interativos
- **MDN Web Docs** — Constraint Validation API
Validação nativa do HTML5 como complemento
- **javascript.info** — Introduction to browser events
Tutorial em profundidade sobre o modelo de eventos

Dica de Prática

A melhor forma de aprender eventos é **experimentar no DevTools**. Abra o console, selecione um elemento com `document.querySelector` e adicione listeners em tempo real — sem precisar editar o código!

Experimente também a aba **Event Listeners** no painel Elements do DevTools para ver todos os listeners registrados em qualquer elemento da página.

